S7 搶資源建模：功能說明與建置邏輯

AssemblyCore · Sensor Micro-Sequencer ISA v1.6 r10 模擬器 · 2026-10-29

> **一句話**：Main 與 Peri 兩個 decoder 在硬體上共用 CSR Arbiter 與 Data Xfer，也共用結果槽；S7 把「兩邊會互相等待」、「結果槽的複製與可見需要時間」、「分支看到的旗標有取樣延遲」做成**可由參數打開的模型**。**八個參數全部預設關閉，開著等於完全沒有改變現有行為**；打開之後，兩邊才開始互相等，而且等了多久、誰在等，在時間軸、ISA Top 圖和統計裡都看得到。
>
> **要先說清楚的**：這些是**模型**，不是已經對照 RTL 或 FPGA 驗證過的硬體行為。使用者尚未確認硬體的實際規則（Arbiter 是否真的存在競爭、裁決規則為何），所以預設全關，由量測來決定要不要打開、數值多少。

目錄

1. 要解決什麼問題
2. 參數一覽
3. 模型規則
    1. 共通原則
    2. CSR Arbiter
    3. Data Xfer
    4. 結果槽：複製與可見
    5. 旗標取樣延遲
4. 看得到什麼：停頓事件、標記、統計、ISA Top 圖
5. 實測數據
6. 如何用 FPGA 量測校正每個參數
7. 驗證：測試與它們證明什麼
8. 建置過程中的設計決策與邏輯規則
9. 假設、限制與待確認
10. 實作位置索引

---

1. 要解決什麼問題

ISA Top 架構圖上，Main 與 Peri 各自有 decoder、計時器與狀態旗標，但**下面這些東西是兩邊共用的**：

共用的東西	圖上的位置	S7 之前
Main/Peri CSR Arbiter	兩個 decoder 的 `EXEC_CSR` 都要經過它才到 CSR	各自獨立計時，**從不互相等**
Data Xfer	Peri 的 `EXEC_PUB`（寫結果槽）與 Main 從結果槽的 `MOV` 共用	從不互相卡
result_slot	Peri `EXEC_PUB` 複製 rx_buf 進去，Main 用 `MOV` 取走	發佈的那一刻 `valid` 立刻為 1，複製沒有額外時間
Status Flag	條件來源（計時器、視窗、結果槽）→ 分支能看到	分支看到的就是**當下**的值，沒有取樣延遲

如果硬體上這些都有成本，而模擬器假裝沒有，**模擬出來的 Main 與 Peri 時間就會比 FPGA 上的樂觀**，而且兩邊交會的時機（例如 Main 何時看到結果）會不一樣。S7 的目的是讓模擬器**能夠**表達這些成本，等你量到硬體的實際行為，再把參數填進去。

設計約束（來自使用者，一直有效）：

- 相容是硬性要求：舊檔案、舊測試、舊結果不能因為 S7 而改變。
- 沒有最佳數值時給初始值就好；之後依 FPGA 量測校調。
- 所有影響時間的條件都是**可調參數**。

---

2. 參數一覽

八個參數，全部在「模擬器」分頁的參數表與「快速參數」裡，也可以用 `asmsim --param key=value` 或 `.simparams` 檔設定。**運行中修改立即生效，不中斷。**

參數	群組	單位	預設	範圍	意思
`csr.arbiter_policy`	CSR arbiter	列舉	`none`	none / main_first / peri_first / round_robin	兩個 decoder 的 `EXEC_CSR` 是否共用一個仲裁器，以及**同一週期都想用時誰先**。none = 完全不建模
`csr.arbiter_penalty`	CSR arbiter	ISA 週期	0	0..100000	輪到等待者時的**交接延遲**：仲裁器空閒後，等待者再等這麼久才開始。只有 policy 不是 `none` 才有意義
`datapath.contention`	Data path	開關	關	開／關	Data Xfer 是否被 Peri 的 `EXEC_PUB` 與 Main 從結果槽的 `MOV` 共用
`datapath.contention_cycles`	Data path	ISA 週期	0	0..100000	同上，Data Xfer 的交接延遲
`slot.copy_extra`	Result slot	ISA 週期	0	0..100000	`EXEC_PUB` 把 rx_buf 複製進結果槽**多花的週期**：指令變長，結果也晚這麼久才可見
`slot.valid_delay`	Result slot	ISA 週期	0	0..100000	從複製完成到 `PERI_RESULT_VALID` 對 Main 可見之間的延遲
`flag.sample_delay_main`	Status flags	ISA 週期	0	0..100000	Main 的 BR 看到的旗標是「這麼多週期以前」的值
`flag.sample_delay_peri`	Status flags	ISA 週期	0	0..100000	同上，Peri 的 BR

預設值的意義：全部為 0／關閉＝**不建模**。測試強制驗證了：把這八個參數明確寫成它們的初始值，跑出來的結果和完全不設定**逐週期相同**（見 §7）。

---

3. 模型規則

3.1 共通原則

這些原則是整個 S7 的骨架，後面每一項都是它們的應用。

原則 1：預設不改變任何結果。 每一條新規則都以「參數不是初始值」為前提；初始值時，程式走的是和 S7 之前完全相同的路徑。

原則 2：等待是「不開始」，不是「做完再補時間」。 一個指令如果需要的共用資源被占用，它**根本不執行**：不產生任何效果、不發出 Issue 事件，只是把它的就緒時刻（`readyAt`）推到輪到它的那一刻，然後在那一刻重新嘗試。 為什麼不能「照常執行、事後加長延遲」？因為指令的**效果發生在執行的那一刻**（例如 `EXEC_CSR` 寫入 CSR

那一刻，可能啟動計時器）。若效果提前發生、只是指令「看起來」比較長，Main 寫 CSR 的時間就和硬體不一致，下游所有時間（計時器到期、事件）都會偏。

原則 3：一個資源有「占用者、空閒時刻、下一位」。 每個共用資源（CSR Arbiter、Data Xfer）各自維護：

- `freeAt`：目前（或最後一個）占用者放手的週期
- `next`：已被授權「下一個輪到」的 decoder（`−1` 表示沒有）
- `grantAt`：它的輪次開始的週期（＝`freeAt` + 交接延遲）
- `last`：最後一個使用者（給 round_robin 用）

原則 4：資源被占用時，第一個來等的人「被授權為下一位」；另一個 decoder 如果在這之前回來，排在他後面。 這條規則是修過兩次才定下來的（見 §8），沒有它會餓死或活鎖。

原則 5：決定性。 同樣的程式、同樣的參數，永遠得到同樣的週期數。模擬器是單執行緒事件驅動；同一週期兩個 decoder 同時就緒時，順序由 `sim.tie_order` 決定，S7 只在「兩個都要 `EXEC_CSR`」這一種情況下，改由 `csr.arbiter_policy` 決定。

原則 6：狀態屬於模擬器的快照。 資源的占用、等待計數、旗標的前一個值都放在 `State` 裡，所以**倒回（rewind）**、**歷史快照**、**重播**都自動一致；不需要另外保存。

原則 7：參數可以在運行中改。 資源的設定（policy、penalty…）在每次套用參數時重新讀取，不重置狀態；改了以後，下一次要用資源的指令就照新設定。

3.2 CSR Arbiter

誰用它：兩個 decoder 的 `EXEC_CSR`（opcode 0），無論 Main 或 Peri、單位元組或多位元組。其他指令（MOV、ALU、BR、`EXEC_XFER`、`EXEC_PUB`）不經過它。

占用多久：一個 `EXEC_CSR` 在**存取的那一段**占用仲裁器。指令總延遲是 1（取指令）＋存取週期，所以占用時間取 `max(1, 延遲 − 1)`。這是一個近似：真正的硬體可能是整個存取期間，或只有位址階段。這個近似寫在 §9。

閘門（指令執行前）：

```
gate(decoder d, 指令是 EXEC_CSR):
    若 csr.arbiter_policy == none:          直接執行
    等待時刻 = claim(arbiter, d, csr.arbiter_penalty)
    若 等待時刻 == 0:                       執行（執行後 occupy）
    否則:  readyAt[d] = 等待時刻;  記一次停頓;  不執行
```

claim：誰在什麼時候輪到

```
claim(資源 r, decoder d, 交接延遲 penalty):
    若 r.next == d:                          # 我之前被授權，現在重試
        若 現在 >= r.grantAt:  r.next = 無;  回傳 0（輪到我了）
        否則:                                回傳 r.grantAt
    若 r.next 是另一個 decoder:              # 對方是下一位，我排在他後面
        回傳 r.grantAt + 1
    若 r.freeAt > 現在:                      # 被占用，我是第一個來等的
        r.next = d;  r.grantAt = r.freeAt + penalty;  回傳 r.grantAt
    否則:                                    # 空閒
        回傳 0
```

occupy（指令執行後）：`arbiter.freeAt = 現在 + max(1, 延遲 − 1)`，`arbiter.last = d`。

同一週期撞上時誰先：排程器每次挑「最早就緒的動作」，同一週期兩個 decoder 都就緒時，原本由 `sim.tie_order` 決定。S7 加了一個例外：**如果兩個 decoder 的下一條指令都是 `EXEC_CSR`，改由 policy 決定**：

policy	同一週期都要 EXEC_CSR 時
`main_first`	Main 先
`peri_first`	Peri 先
`round_robin`	**上一個沒用過的**先（第一次沒人用過，Main 先，之後輪流）

一個重要的結果（不是 bug）：在先到先得、占用時間固定的模型裡，**策略只決定「第一次撞上」誰先**。第一次之後，兩邊會自動**錯開並輪流**（每個人都剛好等到對方放手），所以穩態下兩邊指令數幾乎相同（§5 的表）。策略會影響穩態的情況，是兩個 decoder 的就緒時刻持續恰好相同，這在大多數程式裡不會發生。

3.3 Data Xfer

誰用它：

- Peri 的 `EXEC_PUB`（把 rx_buf 複製進結果槽）
- Main 的 `MOV`，**而且來源是** `PERI_RESULT_SLOT`（讀取結果）

其他 `MOV`（暫存區、ALU 運算元之間）不經過 Data Xfer。這個選擇對應設計文件的描述「MOV 與 result_slot／EXEC_PUB 共用」中最具體、最不會有爭議的那兩種。

占用多久：整個指令的延遲（`EXEC_PUB` 是 `pub(len) + copy_extra`，從結果槽的 `MOV` 是 `mov(len, slot)`）。

規則：和 CSR Arbiter 完全一樣的 `claim／gate／occupy`，只是資源是 `xfer`，交接延遲是 `datapath.contention_cycles`，開關是 `datapath.contention`。

怎麼偵測 Main 的 `MOV` 是不是從結果槽讀：在指令執行**之前**，用 MOV 描述子的 `srcRegion == 2`（結果槽）判斷。這不能等指令執行後才知道，因為原則 2 要求等待發生在執行之前。

3.4 結果槽：複製與可見

現有模型裡，`EXEC_PUB` 執行的那一刻，`valid` 就變 1，結果內容同時寫進槽裡（指令本身的延遲只是 decoder 之後才能發下一條）。S7 在**保持這個行為為預設**的前提下，加兩個延遲：

- `slot.copy_extra`：
  1. `EXEC_PUB` 的延遲 + `copy_extra`（指令變長）
  2. 結果「可見」的時刻晚 `copy_extra`
- `slot.valid_delay`：結果可見的時刻再晚 `valid_delay`

可見時刻：`validAt = 發佈時刻 + copy_extra + valid_delay`，存在結果槽的狀態裡。 **Main 看得到的** `valid`（`slot.valid` 這個計算信號，也就是 `PERI_RESULT_VALID` 條件讀的）：

```
slot.valid（Main 看到的） = 原始 valid  且  現在 >= validAt
```

三個細節：

1. **Overrun 判斷用原始的** `valid`：Peri 再次發佈時，只要槽還沒被清掉就是 overrun，**不管 Main 看到了沒有**。硬體上槽是不是滿的，和 Main 看到的時刻無關。
2. **從結果槽 `MOV` 的檢查用 Main 看到的** `valid`：在 `validAt` 之前 `MOV` 屬於「槽裡還沒有可取的結果」，維持原來的故障（`fault_slot_not_valid`）。這是正確的：硬體上 Main 在看到之前不該去取。
3. **直接寫 `slot.valid` 信號**（環境檔注入、手動設定）時，`validAt = 現在`（立即可見），不受延遲影響。

3.5 旗標取樣延遲

`flag.sample_delay_main` / `flag.sample_delay_peri`：該序列器的 **BR 評估條件時，看到的是「這麼多週期以前」的旗標**。

怎麼做到：

- 每個**儲存型信號**（計時器的 done、輪詢旗標、各種 pending 等）記兩樣東西：**變化前的值**（`sigPrev`）和**變化的時刻**（`sigAt`）。
- BR 評估條件時，把「視野延遲」`viewDelay_` 設成該序列器的延遲。信號讀取時：**如果信號最近一次變化距離現在小於視野延遲，回傳變化前的值**，否則回傳目前值。
- **時間推導出來的信號**（垂直／水平消隱視窗）改用 現在 − 視野延遲 來算，所以視窗「晚開、晚關」。
- **結果槽的可見**（§3.4）也加上視野延遲：現在 `>= validAt +` 視野延遲。
- 評估完立即把視野延遲設回 0，所以**只有 BR 的條件評估**受影響，其他任何讀信號的地方（除錯面板、`until` 條件、環境檔表達式）看到的都是當下值。

範圍與限制：

- **不包含 ALU 旗標**（`ALU_CMP_TRUE` 等）：它們是指令執行的直接結果，不是被取樣的外部條件。
- 每個信號**只保留一個歷史值**：如果在一個取樣延遲之內變了兩次，BR 看到的是倒數第二個值，不是時間上精確的歷史。對 `0`／`1` 旗標足夠，對計數類的信號是近似。
- 事件的檢查點（`event_safe` BR 之後是否派送）不受取樣延遲影響。

---

4. 看得到什麼：停頓事件、標記、統計、ISA Top 圖

一個指令因為資源被占用而不開始，會發出一個 **Stall（停頓）事件**：哪個 decoder、哪個指令（pc）、等什麼（`csr` 或 `xfer`）、要等幾個週期。

地方	看到什麼
時間軸	橘色的小倒三角標記（勾選框「等共用資源」，預設開）；游標停上去：「Peri 等了 6 週期，等的是另一個正在使用的 CSR 仲裁器（週期 N）」；點它跳到那條指令（積木程式就是那個積木）
時間軸的統計行	「等共用資源：Main 333 次（共 N 週期）、Peri 334 次（共 N 週期）」
ISA Top 圖	**CSR Arbiter** 方塊：`busy` 燈、`Main waited`、`Peri waited` 的計數；**Data Xfer** 方塊：`busy` 燈；各方塊說明文字寫明由哪個參數控制
事件面板／匯出	事件類別 `stall`，文字／CSV／VCD 匯出都有
計算信號（可寫進環境檔條件、除錯表達式）	`arb.busy`、`xfer.busy`、`arb.stall_main`、`arb.stall_peri`

「**被停頓**」**怎麼計數**：每次一個指令被擋下來、重新排到之後，算一次；一個指令如果被擋兩次（先排在對方後面，再等對方放手），算兩次。所以「次數」是**被擋下的次數**，不是「等待的事件數」；**週期數**則是累加每次被擋下時要等的週期。

---

5. 實測數據

以下數據都用 `asmsim` 實際跑出來，**可重現**（命令在每張表下面）。

5.1 CSR Arbiter

程式：Main 迴圈 `EXEC_CSR WRITE8 A,1 + JMP`；Peri 迴圈 `EXEC_CSR WRITE8 B,1 + JMP`，兩邊同時開始。把 `latency.csr_access` 設成 6，讓一次存取占 6 週期、迴圈 8 週期，兩個迴圈放不下，會一直在仲裁器上相遇。跑 4000 週期，指令數包含 `EXEC_CSR` 和 `JMP` 兩種。

設定	Main 指令數	Peri 指令數	Main 被擋	Peri 被擋
不建模（預設）	1001	1001	0	0
`main_first`	667	666	333	334
`peri_first`	666	667	334	333
`round_robin`	667	666	333	334
`main_first` + penalty 3	445	444	444	445
`main_first` + penalty 10	251	250	250	250

怎麼讀：

- 預設時兩邊各自全速，互不影響。
- 一旦共用，**總吞吐掉了三分之一**（兩個迴圈共 2002 → 1333 條指令）；先到先得，所以兩邊**輪流**，指令數幾乎一樣。
- 策略的差別只在「第一次誰先」：`main_first` 的 Main 多一條、`peri_first` 反過來；`round_robin` 第一次 Main 先，之後輪流。
- 交接延遲 penalty 讓兩邊**均勻**地更慢（3 週期：889；10 週期：501），**沒有一邊被餓死**。

重現：`asmsim arb.asm --param latency.csr_access=6 --param csr.arbiter_policy=main_first --param csr.arbiter_penalty=10 --cycles 4000 --events ev.csv --dump`

5.2 Data Xfer

程式：Peri 迴圈 `EXEC_PUB service=1, len=4`；Main 迴圈等 `PERI_RESULT_VALID`，再 `MOV PERI_RESULT_SLOT[0..3] -> MAIN_SCRATCHPAD[0..3]`。跑 6000 週期。

設定	發佈	取走	Overrun	Main 被擋	Peri 被擋
不建模（預設）	429	428	429	0	0
`datapath.contention` 開，交接 0	462	461	0	462	462
開，交接 15 週期	143	143	0	143	286

怎麼讀：不建模時，Peri 發佈的速度超過 Main 取走的速度，一半以上的發佈都 **overrun**（結果被丟掉）；打開 Data Xfer 的爭用後，兩邊在這條路上**輪流**，不再 overrun，但每一次都要等；交接延遲越大，吞吐越低。**同一個程式，因為硬體假設不同，會得到不同的結論**，這正是需要用 FPGA 量測來決定模型的原因。

5.3 結果槽與旗標取樣

（時間為「Main 看到並寫出 CSR 的週期」相對於預設的位移，來自測試）

設定	位移
`slot.valid_delay` = 50	**+50**
`slot.copy_extra` = 10	**+10**
兩者同時	**+60**
`flag.sample_delay_main` = 30（旗標在週期 100 變化）	**+30**（容許 ±2，Main 每 2 週期輪詢一次）
`flag.sample_delay_peri` = 40	**+40**（Peri 的 BR）
`flag.sample_delay_peri` = 30，Main 看 `MAIN_TIMER_DONE`	**0**（Peri 的延遲不影響 Main）

---

6. 如何用 FPGA 量測校正每個參數

每個參數在目錄裡都有「量法」。整理如下：

參數	在 FPGA 上怎麼量
`csr.arbiter_policy`	兩個 decoder 在**同一個週期**都發 `EXEC_CSR`，看哪一個的寫入先到 CSR
`csr.arbiter_penalty`	兩邊在迴圈裡持續搶 CSR，量輸家完成時間與贏家的差，減掉贏家占用仲裁器的時間
`datapath.contention`、`datapath.contention_cycles`	Peri 連續 `EXEC_PUB` 的同時，Main 反覆從結果槽 `MOV`，看取走的速率和發佈的速率是否互相影響
`slot.copy_extra`	量 8 位元組 `EXEC_PUB` 的實際時間，和模型的 `pub(len)` 比
`slot.valid_delay`	Main 輪詢 `PERI_RESULT_VALID`，量 `EXEC_PUB` 到 Main 第一次看到之間的週期
`flag.sample_delay_main/peri`	讓一個條件翻轉，找出第一個看得到它的 BR 在翻轉後第幾個週期

流程（建議）：先用預設（不建模）跑一遍，把 FPGA 的時間軸與模擬的時間軸疊起來；差在哪一類（Main 等結果比預期久？Peri 的寫入被延後？），就打開對應那一項，把量到的數值填進去，狀態標成 `measured`，在備註寫量測方式。目標檔（`.simparams`）存進專案，之後所有程式都可以用同一組。

---

7. 驗證：測試與它們證明什麼

核心測試 `asmcore_sim_tests`（217 項，S7 相關的部分）：

測試	證明什麼
所有參數寫成初始值，跑出來和不設定相同	**預設不改變任何結果**（原則 1）
目錄裡沒有「未啟用」的參數	八個參數都被模擬器使用
不建模：兩邊各自跑，互相不等	預設行為
`main_first`／`peri_first`／`round_robin` 第一個週期誰的 CSR 先寫入	同週期撞上的策略
持續爭用後兩邊輪流，差不超過 2 條	先到先得的穩態
共用仲裁器使總指令數下降	共用的成本
penalty 讓輸家更慢，**兩邊都 > 200 條且差 ≤ 3**	交接延遲不餓死任何一邊（回歸測試，見 §8 錯誤 2、3）
penalty 3 比 penalty 10 便宜	單調
只有 penalty 沒有 policy：不變	`none` = 不建模
Data Xfer：關時 Main 取走速度和 Peri 發佈相當；開時 Main 被擋、取走變少	爭用的方向與大小
`slot.valid_delay` = 50 → +50；`slot.copy_extra` = 10 → +10；兩者 → +60	兩個延遲的可加性
在槽可見之前 `MOV` 讀槽 → 故障	Main 看不到就不能取
`copy_extra` 讓 Peri 的 `EXEC_PUB` 指令本身變長	指令延遲
Main／Peri 旗標取樣延遲各自 +30／+40；對方的延遲不影響我	取樣延遲與獨立性
消隱視窗對有延遲的 BR 晚開	時間推導的信號也延遲
全部關閉：`arb.busy`、`xfer.busy`、被擋計數都是 0	沒有任何東西被建模

IDE 服務層 `acide_services_tests`（402 項）：停頓事件 → 時間軸標記 → 統計的對應；用真實的爭用程式跑，確認統計裡 Main、Peri 都有被擋（> 100 次），不建模時是 0。

IDE 介面測試：參數表裡沒有任何「尚未使用」的參數；ISA Top 圖的每個方塊放得下它的列、方塊互不重疊（含新增的 Arbiter 三列）。

---

8. 建置過程中的設計決策與邏輯規則

這一節是**怎麼想的**。每一條決策都附上為什麼，以及（如果有）我走錯過什麼。

8.1 決定：先做機制、預設全關，不等硬體答案

情況：設計文件把 S7 定義為「依硬體確認的結果打開」，但使用者還沒回答硬體上 Arbiter 是否真有競爭、規則為何。 **決定**：**把機制做出來，全部由參數控制，預設關閉**，語意明確寫出，標註「模型，待確認」。 **理由**：預設關閉不改變任何現有結果；同時使用者可以直接做實驗，用 FPGA 量測來校正，而不是等一個結論。**如果硬體答案和模型不同，改的是閘門的規則，介面、參數、顯示都不用動。**

8.2 決定：等待放在「指令執行之前」

考慮過的做法：指令照常執行，之後把它的延遲加長。 **否決的理由**：指令的**效果**發生在執行的那一刻。`EXEC_CSR` 寫 CSR 可能啟動計時器；如果效果提前發生、只是延遲看起來變長，下游一切（計時器到期、事件派送）的時間都和硬體不一致。 **採用的做法**：資源被占用 → 指令**不執行、不發 Issue 事件**，把就緒時刻推到輪到它的時間，到時重試。這和模擬器裡既有的「`EXEC_PUB` 遇到 Main 同週期剛清掉的槽，等一個週期」是同一個手法。

8.3 決定：策略只影響排程器的「同週期撞上」，不另做優先權佇列

考慮過的做法：給每個資源一個優先權佇列，每個請求排隊。 **否決的理由**：只有兩個 decoder，每個同一時刻**至多一個**指令在等；佇列最長是 1。一個「下一位」欄位就夠了，更簡單，也不會藏錯。 **採用的做法**：同週期都要 `EXEC_CSR` 時，排程器改按 policy 排；其他一切沿用既有的 `tie_order`。

8.4 錯誤 1：測試假設會持續爭用，其實不會

我以為：兩個 `EXEC_CSR` 迴圈一直互相擋，兩邊指令數會有明顯差距。 **實際上**：一次 `EXEC_CSR` 只占仲裁器 2 週期，迴圈 4 週期；**第一次撞車之後，兩邊自動錯開，再也不衝突**，被擋總共只有 1 次。 **這是對的**（先到先得的仲裁器就是這樣），錯的是我的測試。 **修正**：把 `latency.csr_access` 設成 6，讓一次存取占 6 週期、迴圈 8 週期，兩個迴圈放不下，才會持續相遇（§5.1）。 **教訓**：寫測試前，先用幾行手算確認「我要驗證的現象真的會出現」。

8.5 錯誤 2：交接延遲讓一個 decoder 餓死

第一版：等待者的就緒時刻 = 資源空閒時刻 + penalty；等待期間**沒有保留資源**。 **實測**：penalty 設成 3 或 10 時，**Peri 一條指令都執行不了**（Main 1001／Peri 0）。 **原因**：Main 的迴圈很短，總是在 Peri 醒來**之前**重新要資源，而資源這時是空閒的，Main 就拿走了；Peri 醒來發現又被占，再等，永遠輪不到。 **教訓**：交接延遲是**已經授權給等待者之後**的開銷；授權之後，別人不能插隊。

8.6 錯誤 3：預留的做法本身造成活鎖

第二版：等待者一進入等待就把 `freeAt` 延到「空閒 + penalty」當作預留。 **實測**：變成 Main 2 條／Peri 0 條，**兩邊都在等，誰也走不了**。 **原因**：沒有「誰擁有預留」的概念。等待者的預留被對方重試時的閘門**再往後推**（因為它看到資源「被占用」），兩邊輪流把 `freeAt` 往後推，永遠推下去。 **最終設計**（§3.1 原則 3、4）：資源有明確的**占用者／空閒時刻／下一位／輪次時刻**；第一個來等的被授權為「下一位」，只有它在輪次時刻之後能拿；另一個如果在這之前回來，排在「下一位」後面，**而且不會改動「下一位」的預留**。 **驗證**：回歸測試要求 penalty = 10 時兩邊都超過 200 條指令，差不超過 3（§7）。

8.7 決定：「預設不變」要有可執行的證明

不只是「我沒有碰預設路徑」，而是一條測試：**把每個參數都明確寫成它的初始值，跑 3 萬週期，結果（時間、指令數、結果槽、內容）必須和完全不設定相同。** 這條測試

在 S7 加了八個參數之後仍然通過，而且原本那條「未啟用的參數不影響結果」的測試，在 S7 把它們全部啟用後，改成了「目錄裡沒有未啟用的參數」。

8.8 決定：資源的狀態放在 `State`，不放在模擬器的成員裡

倒回（rewind）靠快照，快照只保存 `State`。若把「資源何時空閒」放在 `Impl` 的成員，倒回之後它們會和時間不一致，同一個程式倒回再往前跑會得到不同結果。放在 `State` 裡，快照、倒回、重播自動一致，這也是為什麼被擋次數和旗標的歷史值也放在 `State`。

8.9 決定：旗標取樣用「回傳前一個值」而不是真的保存歷史

考慮過的做法：每個信號保存完整的變化歷史，BR 評估時查「現在 − 延遲」那一刻的值。 **採用的做法**：只保存**前一個值**和**變化的時刻**，延遲內回傳前一個值。 **理由**：旗標是 0／1，延遲通常遠小於兩次變化的間隔；省下記憶體和每次評估的查表。**限制寫在 §3.5**（延遲內變兩次，看到的是倒數第二個值）。

8.10 決定：把「是哪個指令用哪個資源」判斷放在指令執行之前，用 opcode

CSR Arbiter：opcode 0。Data Xfer：Peri 的 opcode 2，以及 Main 的 MOV 且描述子的來源區域是結果槽。 這不能依賴「指令執行後才知道的資訊」，因為等待要在執行之前（§8.2）。為此加了一個只讀的小函式（`mainMoveFromSlot`）解析 MOV 描述子，不執行任何效果。

8.11 規則：看得到才信得過

每個新行為都有三個對應的「看得到」：

1. 一個事件（Stall）進事件流，所以時間軸、事件面板、匯出都有；
2. 計算信號（`arb.busy`、被擋計數），所以 ISA Top 圖與除錯表達式都能讀；
3. 統計行的數字。

理由：校正的前提是看得見差異。一個建模了卻看不見的行為，無法用來和 FPGA 比對。

---

9. 假設、限制與待確認

這些是我選的假設，不是硬體確認過的事實：

1. **占用時間** = `max(1, 延遲 − 1)`（CSR Arbiter）：假設仲裁器在指令的存取段被占用，取指令那一週期不占。真實的占用可能是整個存取、只有位址階段，或和延遲無關的固定值。
2. **Arbiter 只管** EXEC_CSR：EXEC_XFER 的 I2C 傳輸路徑（i2ch、i2cm_top）有自己的延遲參數（`i2ch.overhead`、`i2c.*`），不經過這個仲裁器。如果硬體上 EXEC_XFER 也要先經過 Arbiter，這一項要加。
3. **Data Xfer 的使用者只有兩種**：Peri EXEC_PUB 與 Main 從結果槽的 MOV。其他 MOV 是否也經過，未確認。
4. **先到先得**：資源被占用時，第一個等的人被授權為下一位。如果硬體是固定優先權（例如 Main 永遠優先，在它想要時可以插隊），那是另一種規則，目前沒有。
5. **交接延遲只算在「等過」的那一方**：沒有等的指令不加交接延遲。
6. **旗標取樣不含 ALU 旗標，且只保留一個歷史值**（§3.5）。
7. **事件派送的檢查點不受取樣延遲影響**。

待你確認的硬體問題（和設計文件 §8 對應）：

- CSR Arbiter 與 Data Xfer 在硬體上**是否真的存在競爭**？規則是什麼（先到先得／固定優先／輪流）？
- 一次存取占用多久？交接有沒有額外延遲？
- 結果槽的 `valid` 在複製完成之前能不能被 Main 看到？
- 條件旗標到 BR 之間有幾個週期的延遲？

---

10. 實作位置索引

內容	檔案與函式
八個參數的定義、說明、量法	`core/src/sim_params.cpp`（群組 CSR arbiter／Data path／Result slot／Status flags）
狀態：資源占用、等待計數、信號歷史、結果槽可見時刻	`core/src/sim/impl.h`：`State::SharedRes`、`stalls`、`stallCycles`、`sigPrev`、`sigAt`、`SlotState::validAt`
參數讀取（含運行中套用）	`core/src/sim/impl.cpp`：`applyParams`
閘門、輪次、占用、同週期策略	`core/src/sim/run.cpp`：`claim`、`gate`、`occupy`、`nextIsCsr`、`mainMoveFromSlot`、`nextTime`
兩個 decoder 的接入點	`core/src/sim/main_core.cpp`：`stepMain`；`core/src/sim/peri_core.cpp`：`stepPeri`、`opPeriPub`
結果槽的可見與取樣延遲	`core/src/sim/impl.cpp`：`slot.valid` 計算信號、`sigGet`、`sigSet`、`viewNow`、`slotVisible`、`evalCondition`
Stall 事件	`core/include/asmcore/sim.h`：`EventType::Stall`
時間軸標記與統計	`AssemblyCoreIDE/services/sim_compare.cpp`：`markerOf`（kind `z`）、`traceStats`
時間軸顯示	`AssemblyCoreIDE/ui/sim_panels.cpp`：`ExecutionView`（橘色三角、「等共用資源」勾選框、統計行）
ISA Top 圖	`AssemblyCoreIDE/services/isa_layout.cpp`：`arb`、`xfer` 方塊的列
核心測試	`tests/sim_tests.cpp`：S7 區塊
服務層測試	`AssemblyCoreIDE/tests/services_tests.cpp`：`t_trace_links` 的 S7 區塊
參數文件（自動產生）	`docs/SIM_PARAMS.md`（`gen_sim_params_doc`）